Principes des Systèmes et Réseaux

Emilie Morvant

Laboratoire Hubert Curien
Université Jean Monnet de Saint-Etienne
Faculté des Sciences et Techniques

Licence 2 - Semestre 4

2015 - 2016

Qui suis-je ?

- Maître de Conférence

  à l'Université Jean Monnet, St-Etienne, France
  au Laboratoire Hubert Curien
  dans l'équipe d'apprentissage automatique
  responsable de la Licence 2 d'informatique

- Domaine de recherche :
  - Domaine principal : Apprentissage Automatique *(Machine Learning)*
  - Théorie de l'apprentissage automatique statistique
  - Applications à des pbs de vision par ordinateur *(Computer Vision)*

- Comment me joindre :

  Mail : `emilie.morvant@univ-st-etienne.fr`

  Twitter : @EmilieMorvant

Objectifs du cours

- Connaître le fonctionnement général d'un système d'exploitation *(OS : Operating System)*
- Connaître les principaux composants d'un système d'exploitation
- Être capable d'utiliser efficacement la ligne de commande Unix
- Écrire des scripts bash

Référence : *Systèmes d'exploitation* (3ème Ed.) de Andrew Tanenbaum

Généralités

Un peu d'architecture des ordinateurs

Les composants d'un ordinateur

- processeur(s) (CPU)
- memoire vive (RAM) (volatile : besoin electricité)
- péripheriques d'entrées-sorties
- memoire de masse (disques durs, CD, DVD)
- écran
- souris, clavier
- réseau
- etc.

Un peu d'architecture des ordinateurs

Ordinateur

Un ordinateur est un calculateur programmable
i.e. un (ou plusieurs) processeur(s) qui effectuent des calculs suivant une liste d'instructions (un programme)

Programme, données et résultats des calculs : stockés dans la mémoire vive

Système d'exploitation

Interface entre les applications et le matériel physique

Fonctionnement de l'ordi : tâche non interactive

Exemple : Prévision météo

- **Données** : mesures de températures, pression, etc. du jour
- **Programme** : simulation météo
- **Résultat** : pévisions sde températures, pression etc. 24h plus tard

3 étapes :

1. Chargement des données et du programme ds la RAM (depuis le disque)
2. Exécution du programme par le CPU
3. Sortie des résultats (sur écran, disque, imprimante, etc.)

Non interactivité

L'utilisateur n'intervient que au début et à la fin

Important : La prévision doit se faire en moins de 24h

Fonctionnement de l'ordinateur : tâche interactive

Exemple : Jeu vidéo

- **Données** : données du jeu et actions du joueur (clavier, souris)
- **Programme** : calcule les conséquence des actions du joueur
- **Résultat** : une image et du son

Les étapes :

1. chargement données et du programme
2. exécution du programme
3. actions du joueur
4. exécution prog
5. etc.

Important : Fluidité et réactivité du programme

Fonctionnement de l'ordinateur : en général

L'ordinateur est **piloté** par le(s) processeur(s) (CPU *central processing unit*)
qui **exécute(nt)** les instructions du programme l'une après l'autre
qui est **cadencé** par une horloge qui envoie des 'top'

Remarques

- Temps entre 2 'top' = 1 cycle d'horloge
- Dans les années 80 : 1-4Mhz
- **Now : 1.5-3.5 Ghz** (3Ghz = 3 milliards opérations à la seconde)
- En général 1 top ⇒ une instruction exécutée

Fonctionnement de l'ordinateur : en général

Les instructions possibles

- Lecture/Écritures des données en RAM
- Faire des calculs :
  - ↪ addition, soustraction, multiplication, divisions, comparaisons, etc.
- Modifier l'endroit où sera lu la prochaine instruction en fonction d'une comparaison
  - ↪ En C : boucles ou `if else`
- Instructions d'Entrée/Sortie
  - ↪ envoyer ou recevoir des données d'un périphérique d'E/S

Un peu d'histoire

Années 45-55 : **Tubes à vide**

- Gros ordinateurs très chers (1 terrain de foot, quelques ordi dans le mondes)
- Calculs scientifique (ex : tables de sinus)
- Utilisation très lourde : Programmer $\Leftrightarrow$ Tableau d'interrupteurs

Un peu d'histoire

Années 55-65 : **Transistors**

- Un ordinateur par (grande) université/entreprise
- Carte perforées
- Traitement par lots (batch) : une tâche après l'autre
- Non interactif, pas réellement de SE

Exemple : diagramme d'utilisation

- 1 tâche à la fois
- Un ordi avec 1 CPU, 1 entrée (lecteur de bande), 1 sortie (imprimante)
- Chargement en mémoire : C, Execution sur CPU : E, sortie des résultats sur Imprimante : I

tâche 1	C	E	I													
tâche 2				C	E	I										
tâche 3							C	E	C	E	I					2 E : toutes les données ne tiennent pas en RAM
tâche 4												C	E	I		
entrée	X			X			X		X			X				
proc		X			X			X		X			X			5/14 d'utilisation
imprim			X			X					X			X		

↪ Gestion simple : 1 seule tâche en RAM

Un peu d'histoire

Années 65-80 : **Circuits intégrés**

- Calculs scientifiques et gestion commerciale

- Traitement par lots avec gestion **multitâche**

↪ Plusieurs tâches sont chargées dans l'ordinateur

  - Quand une fait des E/S, une autre est exécutée par le CPU

- Une seule tâche s'exécute à la fois ⇒ Il faut partager le temps

Exemple : diagramme d'utilisation

- multitâche : plusieurs tâches en RAM
  - ⇒ plus complexe
  - ⇒ besoin d'un Système d'exploitation
- Un ordi avec 1 CPU, 1 entrée, 1 sortie
- Chargement en mémoire : C, Execution sur CPU : E, sortie des résultats sur Imprimante : I

tâche 1	C	E	I						
tâche 2		C	E	I					
tâche 3			C	E	C	E	I		
tâche 4				C	E	I			
entrée	X	X	X	X	X				
proc		X	X	X	X	X	X		5/7 d'utilisation
imprim			X	X		X	X		

Un peu d'histoire

Depuis les années 80 : **Ordinateurs personnels**

- Problème du multitâche : réactivité très faible

↪ Il faut attendre la fin des tâches précédentes

- Systèmes d'exploitations modernes

↪ temps partagé (ou multitâche préemptif)

↪ plus d'interactivité, utilisation de terminaux (1 clavier + un écran)

- temps de réponse — 1 quantum : 10 ms

⇒ tout semble s'exécuter simultanément efficacité un peu plus faible.

Rôles du système d'exploitation (système à temps partagé)

- **Gestion des ressources** : attribuer selon une politique donnée les ressources aux différentes tâches (mémoire, temps processeur, périphériques)
  - Ex : La politique peut favoriser les tâches interactives aux tâches de calcul
- **Gestion des droits / sécurité / respect de la confidentialité** :
  - Une tâche ou un utilisateur ne doit pas pouvoir accéder aux données d'une autre tâche ou d'un autre utilisateur (sauf autorisation explicite)
  - Une tâche ne doit pas pouvoir perturber ou empêcher l'exécution d'une autre tâche (Virus, Vers, attaque D.O.S. Denial Of Service : déni de service)
- **Simplifier la programmation** : indépendance par rapport à la diversité des matériels, utilisation de bibliothèques système
  - Ex : utilisation de fichiers (clé USB, DD, ...)

  ⇒ fournit une interface uniforme indépendante des détails matériels (sous UNIX, tout est fichier : arborescence `/dev/`)

Types de Systèmes d'Exploitation

- **mono-utilisateur / multi-utilisateur** ($\Rightarrow$ login, mdp)
  - Ex : DOS, WIN 95/98, UNIX, GNU/LINUX, téléphone
- **monotâche / multitâche**
- **temps partagé ou préemptif / non préemptif**

SE moderne : multi-utilisateur, multi-tâche, préemptif

Organisation générale d'un SE

Un système d'exploitation contient :

1. un noyau
2. des bibliothèques systèmes
3. des processus système (démons, daemon)
4. des programmes utilitaires indispensables
5. un contrôle d'accès

Un concept clé : le processus

Un processus est un **programme en cours d'exécution**. Un processus a

- un espace d'adressage : ensemble d'adresses mémoires allant de 0 à une limité donnée où il peut lire et écrire
- un ensemble de registres : dont le compteur ordinal, le pointeur de pile, des registres matériels et toute l'information nécessaire pour exécuter le programme

Organisation générale d'un SE

Le noyau du SE dont les principaux composants sont :

- un gestionnaire des processus :
  - choisit le prochain processus à exécuter parmi les processus prêts (ordonnanceur)
  - met les processus en sommeil lors d'une attente d'E/S et les réveille
  - permet aux processus de s'envoyer des signaux, de se synchroniser, de définir des zones d'exclusion mutuelle
- un gestionnaire de mémoire :
  - alloue la mémoire aux différents processus en fonction de leurs demandes et de la quantité de mémoire disponible. Sécurité, isolation des processus
- un accès aux périphériques :
  - mise en route de ceux-ci
  - répond aux requêtes des processus
  - utilise des pilotes de périphériques (drivers) fournis par les fabriquants

Organisation générale d'un SE

Bibliothèque système :

fournit aux processus des fonctions pour accéder aux services du noyau

- Ex : Sur GNU/LINUX : la libc (`stdio.h`, `stdlib.h`, ...), nice, malloc, printf, fopen

processus système (démons, daemon) lancés au démarrage de l'ordi

- serveurs divers : serveur d'impression, serveur ftp, mail, ...
- sous LINUX : gestionnaire de l'affichage graphique de la souris et du clavier (serveur X)

Organisation générale d'un SE

programmes utilitaires indispensables

sans eux, l'utilisation ou l'administration de la machine est impossible

- interpréteur de commandes : shell (sh, bash, csh, ...)
- gestionnaire de fenêtres (window manager)
- utilitaires de gestion des fichiers et répertoires : `ls`, `rm`, `cp`, `mv`, ...
- pour créer des comptes utilisateur et changer les mots de passe
- éditeur de texte (vi, emacs, ...) (indispensable pour l'administration système)
- compilateur (gcc)
- etc.

contrôle d'accès

au moins deux modes : superviseur/noyau et utilisateur (rien a voir avec root/non root)

Quelques mots sur les licences des logiciels

Logiciel propriétaire : licence propriétaires

⇒ Seul le binaire est fourni

⇒ Seulement un droit d'utilisation

⇒ Pas le droit de modifier/redistribuer/étudier le logiciel (pour vendre)

Inconvénients

- Formats de fichier propriétaire

⇒ impossibilité de récuperer les données stockées dans les fichiers

- Sécurité : comment peut on etre sur que le logiciel fait bien ce qu'il pretend faire (en particulier ne fait rien de plus) ?

⇒ pb confiance/sécurité (ex : skype, vote electronique, . . .)

Mais ce n'est pas tout. . .

Quelques mots sur les licences des logiciels

Inconvénients des logiciels propriétaires illustrés sur un exemple

Une entreprise achète un logiciel. Elle met dedans des données clients.
L'entreprise souhaite modifier le logiciel pour :

- ajouter des fonctionnalités
- corriger des bugs
- l'utiliser avec du matériel plus récent ou avec un autre SE
- ...

Que peut faire la société si :

- la société qui a vendu le logiciel a disparu
- elle ne vend plus ce logiciel
- elle ne veut pas le modifier ou a un prix exorbitant
- la nouvelle version du logiciel est trop chère

Quelques mots sur les licences des logiciels

Logiciel libre

⇒ le code source est fourni

⇒ On a le droit de tout faire : l'utiliser, l'étudier, le modifier pour ajouter des fonctionnalités ou corriger des bugs, redistribuer le logiciel modifié, créer un autre logiciel à partir de lui, ...

Un type de licence particulière "héréditaire" : GPL

- tout logiciel produit en modifiant un logiciel GPL est GPL

⇒ donnant/donnant : je te donne mon code, tu me donnes le tien

Remarques

- Les logiciels freeware ou shareware NE sont PAS libres

⇒ ils sont gratuits (enfin presque...)

- Les logiciels "open source" ne sont pas nécessairement libres

⇒ "open source" signifie que les sources sont disponibles

Quelques mots sur les licences des logiciels

Logiciel libre

Inconvénient : On ne peut pas réellement vendre du logiciel libre

Avantages : Aucun des problèmes des logiciels propriétaires

- On peut vendre du service qui va avec (ce que font des petites comme des grandes entreprises : IBM par ex)
- On tire profit de la communauté qui fournit des rapports de bugs, des correctifs, des améliorations
- une entreprise A qui produit du code libre peut utiliser une double licence : GPL / propriétaire :
  - si une entreprise B veut developper du code propriétaire a partir du code de A, elle achète une licence propriétaire qui le permet

Quelques mots sur UNIX-LINUX

Famille des UNIX : Premiers SE modernes (1970)

Projet GNU et la FSF

1983 : il n'existe pas d'UNIX libre

⇒ Richard Stallman propose d'écrire un système d'exploitation UNIX libre

⇒ Projet GNU : **G**nu is **N**ot **U**nix

Depuis cette date, de très nombreux programmes ont été écrits :
utilitaires, compilateur C (gcc), emacs, bibliothèques systeme (libc), ...
Le noyau Hurd... mais toujours pas de version stable...

1985 : création de l'association FSF (Free Software Foundation)

- supporter le "logiciel libre"
- le projet GNU
- GPL : GNU General Public License.

Quelques mots sur UNIX-LINUX

LINUX

1991 : Création de Linux par un étudiant Finlandais, Linus Torvalds
Depuis d'autres noyaux libres : FreeBSD, NetBSD, OpenBSD, ...

système complet : GNU/LINUX est l'UNIX le plus utilisé

Mais, il est difficile d'installer GNU/LINUX

Quelques mots sur UNIX-LINUX

Distribution

Pour faciliter l'installation de GNU/LINUX on passe par une **distribution**

Distributions LINUX (wikipedia)

On appelle distribution LINUX, une solution prête à être installée par l'utilisateur final comprenant le système d'exploitation GNU, le noyau LINUX, des programmes d'installation et d'administration de l'ordinateur, un mécanisme facilitant l'installation et la mise à jour des logiciels comme RPM ou APT ainsi qu'une sélection de logiciels produits par d'autres développeurs.

⇒ Il existe de très nombreuses distributions de LINUX

ex: Debian, Red-hat, Ubuntu, kubuntu, ...

Suite du cours

Un chapitre pour chaque rôle du noyau :

1. Gestion des processus
2. Gestion mémoire
3. Gestion Entrées/Sorties

et deux chapitres sur des E/S particulières :

4. Systèmes de fichiers
5. Réseaux

Gestion des processus

Définitions

Programme

Code décrivant une tâche
Langage machine dans un fichier

Processeur

Automate électronique de traitement
Change son état en fonction des instructions

Processus

Un programme exécuté par un processeur

Analogie

Un processus est un programme en cours d'exécution

Analogie

- CPU : cuisinier
- programme : recette de cuisine
- entrées : ingrédients
- sorties : le plat fini
- processus : l'action de lire la recette et de préparer le plat

ATTENTION

Il ne faut pas confondre processus avec processeur

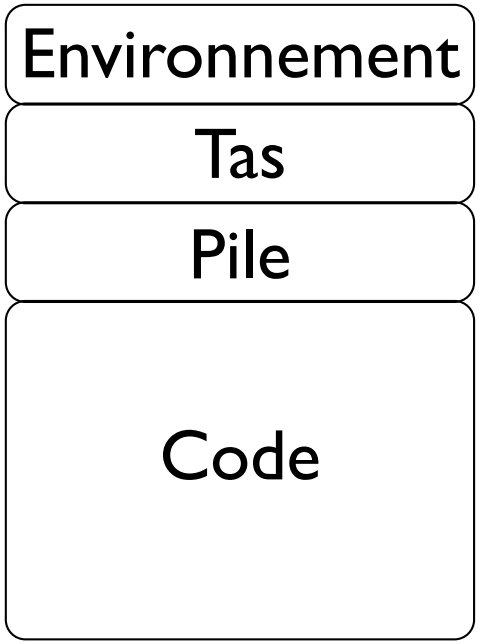
Propriétés des processus

Objectifs

- Séparer les différentes tâches du système
- Gestion des fichiers et des applications
- Permettre le multitâche

$\Rightarrow$ Chaque processus doit respecter les autres !!

Structure d'un processus



- Environnement : variables héritées
- Tas : données globale statiques, variables allouées dynamiquement, partagées par tous les threads du processus
- Pile : Variables temporaires du programme, changements de contexte, limité
- Code : Recopie du programme machine, fixe et protégé en mémoire, segment exécutable du processus

Hiérarchie des processus

À l'exception du processus initial, un processus (le processus fils) est toujours créé par un autre processus (le processus père)

⇒ On a donc une **arborescence** des processus
↪ avec héritage des droits (sauf dans certains cas)

Exemple : login et lancement d'un processus

gestionnaire session → window manager → xterm → shell → cp

Caractéristiques des processus stockées par le SE

- PID *(Process Identification)* : un processus est un numéro. . .
- PPID *(Parent Process Identification)* : le numéro du processus parent
- UID, GID : identifiant de l'utilisateur et du groupe qui a lancé le processus
- tty : terminal d'attachement
- pri : priorité
- s : état du processus
- stime : date de lancement
- time : temps écoulé d'utilisation
- size : taille de la mémoire allouée
- zones mémoires utilisées par le processus (sécurité)
- ligne de commande
- infos pour reprendre le processus (registres CPU)
- etc.

Le cycle de vie d'un processus

Démarrage

- Chargement du code
- Édition des liens dynamiques
- Recopie de l'environnement
- Initialisation du tas
- Lancement de la première instruction

En cours d'exécution

- Exécution instruction par instruction
- Mise en attente si
  - Demande d'E/S a satisfaire
  - Interruptions matérielles
- Allocation des ressources sur demande

Les Threads (processus légers)

Thread

Un thread ou processus léger est une partie des instructions du processus en cours d'exécution.

Un processus peut contenir un ou plusieurs thread s'exécutant en quasi-simultanéité (ou simultanément sur les processeurs multi-cœurs).

À la différence des processus, les threads partagent le même espace mémoire protégé, les mêmes ressources et le même espace mémoire.

Remarques :

- Un Thread est l'unité d'ordonnancement
- Multi-thread : actions simultanées sur des ressources partageables
- Gestion 10 à 100 fois plus rapide que pour les processus

Comment exécuter des programmes en même temps ?

On souhaite exécuter plusieurs programmes en même temps

Par exemple

- Faire travailler plusieurs utilisateurs sur la même machine pour que chacun ait l'impression d'avoir la machine à lui tout seul
- Compiler tout en lisant ses mails

Problème

Un processeur ne peut exécuter qu'une seule instruction à la fois

Solution

Partager un (ou plusieurs) processeur entre différents processus

Commutation de processus

Multitâche

Un SE est **Multitâche** s'il peut exécuter "simultanément" des programmes

Cette simultanéité n'est qu'apparente

↪ C'est le résultat rapide d'exécution de processus

⇒ Il faut ordonnancer des tâches

Commutation de processus

La commutation de processus sauvegarde l'état d'un processus pour restaurer/reprendre (à la place) l'état d'un processus précédemment interrompu dans le cadre de l'ordonnancement d'un SE multitâche

Multitâche coopératif : les programmes initient les commutations

Multitâche préemptif : le SE initie les commutations lors d'évènement externes

Commutation de processus—Un exemple simple

- Un processus P1 en cours d'execution
- Prise d'une "photo" de l'etat de P1
- Choix du nouveau processus (par ex. P2)
- Restauration de l'état de P2 (la "photo" de P2)
- P2 s'exécute
- ...
- Restauration de l'état de P1 (à partir de sa "photo")
- P1 continue à partir de là ou il a été interrompu

Photo de l'état d'un processus $\Longleftrightarrow$ "où on en est dans la recette"

La commutation—Pourquoi faire ?

Pour augmenter l'utilisation des ressources

- Analogie : plusieurs plats à faire (entrée, plat, dessert)
  à certains moment le cuisinier attend que quelque chose cuise (ex : le plat)

⇒ il en profite pour commencer à preparer l'entrée

⇒ la cuisson est finie : il reprend la préparation du plat sans le laisser cramer

Pour l'interactivité

- Presser une touche, traitement de 10ms, 100ms

⇒ peu d'utilisation du CPU

La commutation—Qui et Quand ?

Qui effectue la commutation ? $\Longrightarrow$ C'est le système d'exploitation

- il stocke l'état du processus en RAM
- il choisi le nouveau processus
- il récupère son état et le relance

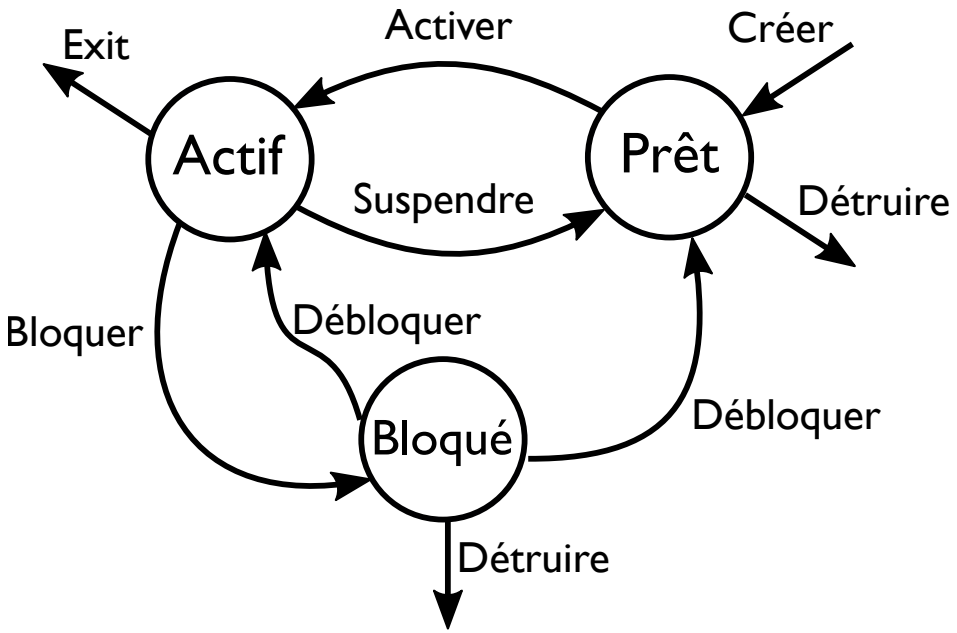
Quand se fait la commutation ? $\Longrightarrow$ Il faut décider quoi exécuter lorsque

- un processus est créé $\rightsquigarrow$ faut-il exécuter le processus parent ou l'enfant ?
- un processus est terminé $\rightsquigarrow$ qui exécuter maintenant ?
- le processus en cours est en attente d'E/S
- sur un système préemptif : l'horloge envoie un signal (top) au processeur pour lui indiquer qu'il est temps de faire une commutation
  N.B. : temps entre 2 "tops" : un quantum

États d'un processus—Graphique des états

Cas multitâche temps partagé

Le graphique des états est représenté par un automate où chaque processus "se deplace" d'un état à l'autre : prêt, exécution/actif, sommeil/bloqué



dans l'état prêt : autant de processus que de "coeurs"

$\hookrightarrow$ machine avec 2 processeurs quadri-coeurs $\Rightarrow$ 8 coeurs

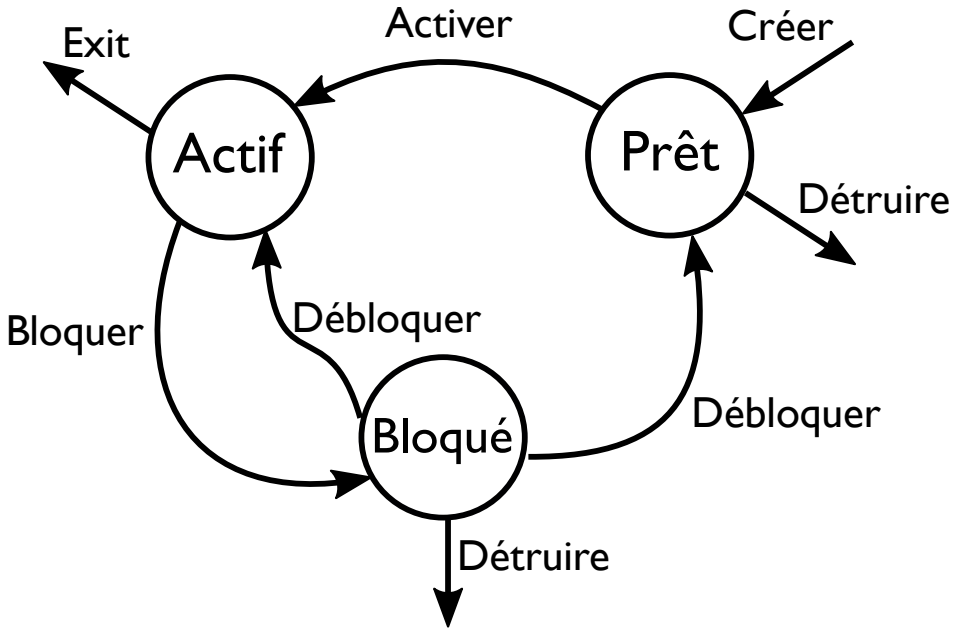
$\Rightarrow$ au plus 8 processus en meme temps

États d'un processus—Graphique des états

Cas multitâche SANS temps partagé

RAPPEL : Le SE ne peut suspendre un processus

⇒ PAS DE TRANSITION entre Actif et Prêt



États d'un processus—Les Transitions

- Création de processus ⇒ mis dans l'état prêt
  - par le SE sous linux : processus init est de PID 1)
  - par un autre processus ⇒ hiérarchie des processus
- Prêt ⟶ Actif/Exécution : choisi par l'ordonnanceur
- Actif/Exécution ⟶ Prêt : fin du quantum
- Exécution ⟶ Bloqué/Sommeil
  - volontaire (sleep)
  - attente E/S (appel bloquant ⇒ sommeil)
  - par un autre processus (bloqué)
- Bloqué/Sommeil ⟶ Prêt
  - volontaire
  - fin E/S bloquant
  - par un autre processus (bloqué)
- Fin de processus
  - par lui-même
  - il peut être tué par un autre processus
  - il peut être tué par le SE en cas d'erreur grave

  Ex : tentative d'execution d'une instruction invalide, accés mémoire invalide (segfault)

Ordonnancement—Les Objectifs

Rôle de l'ordonnanceur

L'ordonnanceur choisi parmi tous les processus et suivant sa politique, le prochain qui va être exécuté $\rightsquigarrow$ notion de priorité

Les objectifs de l'ordonnancement sont :

- **Équité** : les processus doivent recevoir un tps équitable (selon la politique)
- **Équilibre** : toutes les parties du sytème doivent être utilisées
- **Éviter la famine** : éviter qu'un processus ne soit jamais exécuté

Attention il faut faire la distinction entre :

- priorité externe qui est définie par l'utilisateur (variant peu)
- priorité interne qui est gérée par l'ordonnanceur (variant plus souvent)

Ordonnancement—Quelle politique ?

Exemples :

- Optimisation de l'efficacité (utilisation proc/temps) et du temps d'attente moyen (temps entre la soumission de la tâche et du résultat)

⇒ traitement par lots (sans interaction)

  - Ex : prévisions météo, écoulement d'air autour d'une aile d'avion, séquençage du génôme, ...

- Rapidité du temps de réponse

⇒ Systèmes intéractifs

- Garantir autant que possible aux processus certains délais

⇒ Système temps réel

  - Ex : réagir aux évènements extérieurs, monitoring, contrôle de robots (chaine de montage), avions, securité centrale nucléaire, ...

Ordonnancement—Stratégies Typiques

- FIFO First In First Out (sans préemption, algorithme le plus simple)
- Le Tourniquet
- Shortest Next Job
- Highest Response Ratio Next

Ordonnancement—First In First Out

FIFO : Traitement par lots sans application intéractive

- Premier arrivé, premier servi

↪ le SE choisit le processus le premier dans la file d'attente

- Ordonnancement coopératif (pas de préemption) : les processus rendent la main "de leur plein gré"
  - lorsqu'ils se terminent
  - lorsqu'ils se bloquent
  - lorsqu'ils font l'appel système `yield`
    (`yield` permet de céder le processeur aux autres processus)

N.B. Cet algo peut faire appel a des priorités

↪ Le SE choisit le processus prêt de plus haute priorité

Ordonnancement—First In First Out

Inconvénients

- le temps de réponse dépend du processus qui a la main
  - tant que la main n'est pas rendu, les autres processus doivent attendre
- pénalise les processus courts
  - proportion temps d'attente/temps d'exécution

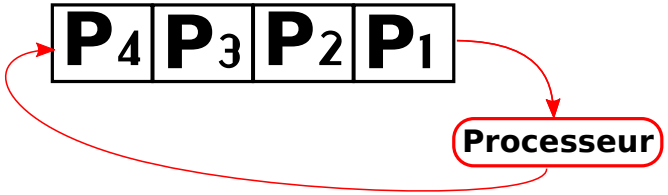
Avantages

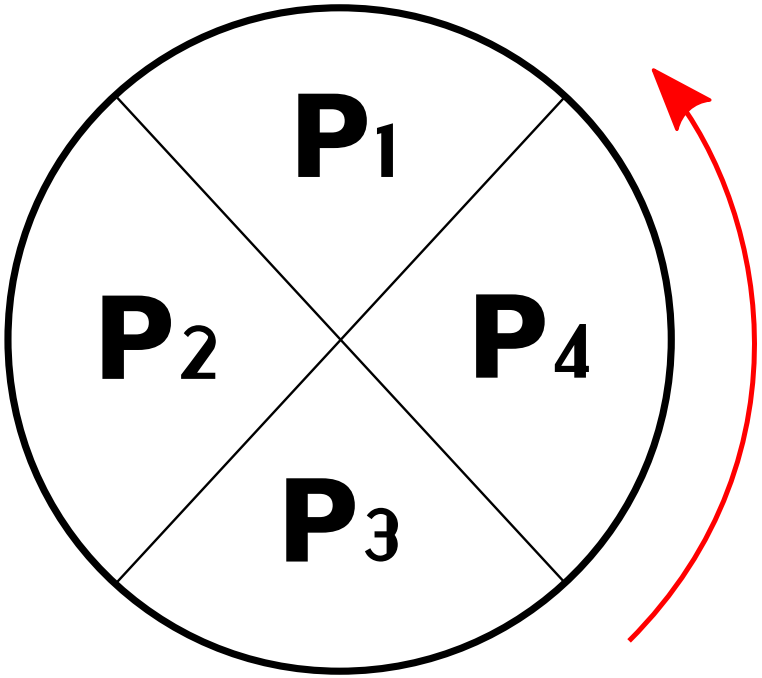
- simple
- surcoût faible
- équitable
- Adapté dans les contextes suivants :
  - nombreux cœurs de calculs
  - processus très fréquemment bloqués (gestions E/S)
  - machine peu puissante, où le surcoût doit être minimisé

Ordonnancement—Tourniquet

Tourniquet *(ruban round)*

- FIFO avec préemption
- Chaque processus reçoit un quantum de temps
- Une fois le quantum épuisé (ou que le processus est bloqué), le processus passe la main et retourne à la fin de la file d'attente





Ordonnancement—Tourniquet

Inconvénients

- très sensible au choix du quantum
  - en diminuant la durée du quantum
    - le temps de réponse diminue, mais...
    - le surcoût augmente
  - le quantum idéal dépend de la durée moyenne d'une intéraction et du nombre de processus...
- défavorise les processus orientés E/S (bloqués avant la fin de leur quantum)

Avantages

- temps de réponse borné indépendamment des processus (calculs ou E/S)
- équitable

Ordonnancement—Shortest Job Next

La main est toujours donnée au processus qui va mettre le moins de temps avant de se bloquer ou terminer

⇒ **Requiert une hypothèse forte**
avoir une connaissance/estimation de ce temps pour chaque processus

Inconvénients

- surcoût
- inéquitable
- famine possible (en particulier pour les processus calculatoires)

Avantages

- maximise le temps de réponse
- maximise le débit (nombre de processus terminés par unité de temps)

Ordonnancement—Highest Response Ratio Next

Variante du Shortest Job Next

↪ prise en compte du ratio du temps que le processus a passé à attendre sur le temps dont il a besoin

Répond au problème de famine :

- plus un processus attend, plus il augmente ses chances d'avoir la main
- mais suppose toujours la connaissance du temps d'exécution

Formalisation de l'ordonnancement

Notion de tâche

Unité élémentaire de traitement ayant une cohérence logique

Un processus est composé de plusieurs tâches : $P = T_0, \ldots T_n$

Chaque tâche inclue une ou plusieurs instructions

Relation de précédence

Une relation $<$ est une relation de précédence sur une ensemble E

- transitive : $\forall (x, y, z) \in E^3,\ (x < y, y < z) \Rightarrow x < z$
- anti-réflexive : $\forall e \in E,\ \neg(e < e)$
- anti-symétrique : $\forall (e, e') \in E^2,\ (e < e', e' < e)\ \Rightarrow\ e = e'$

Formalisation de l'ordonnancement

Système de tâches

Un système de tâche $S = (E, <)$ vérifie les deux propriétés suivantes :

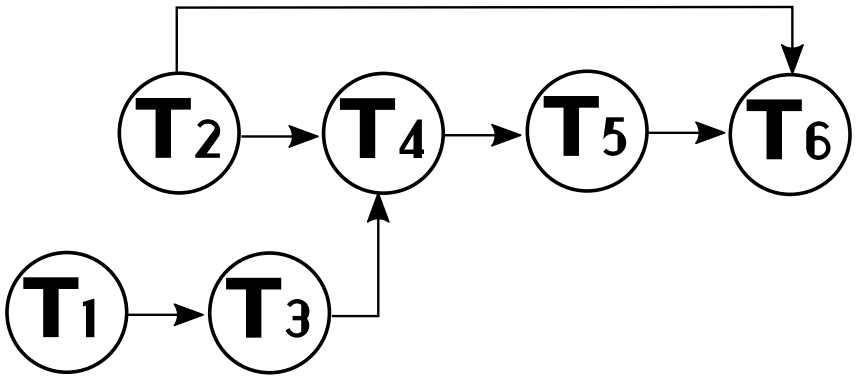
- si $(T_i, T_j) \in E^2$, $T_i < T_j$ représente l'exécution en séquence de T_i puis T_j
- si $(T_i, T_j) \in E^2$, $\neg(T_i < T_j)$ alors T_i et T_j sont exécutables en parallèle

Graphe de précédence

Graphe correspondant à la relation $\rightarrow$ définie par

$$T_i \rightarrow T_j \Leftrightarrow T_i < T_j \text{ et } \forall T_k,\ \neg T_i < T_k < T_j$$

La donnée d'un graphe de précédence équivaut à celle d'un système de tâche



Communication Inter-Processus

Les processus peuvent avoir besoin de communiquer entre eux

- Synchronisation
  - commencer une tâche quand un autre processus a fini la tâche précédente (c.f. graphe de précédences)
- Exclusion mutuelle
  - accès exclusif à une ressource (mémoire, fichier, imprimante, ...)
- Signaux
- Échange de message
- Partage de mémoire

Exemple simple :
dans un pipeline |, la sortie du premier processus doit être passée au second

Gestion de la mémoire

Unité de mesure de taille (Rappel Architecture)

Nom	Symbole	Valeur (en o)	Erreur	Nom	Symbole	Valeur (en o)
octet	o	1	0%	octet	o	1
kilo	ko	$10^3 = 1\,000$	2%	kibi	Kio	$2^{10} = 1\,024$
méga	Mo	$10^6 = 1\,000\,000$	5%	mébi	Mio	$2^{20} = 1\,048\,576$
giga	Go	$10^9 = 1\,000\,000\,000$	7%	gibi	Mio	$2^{30} = 1\,073\,741\,824$
téra	To	10^{12}	10%	tébi	Tio	$2^{40} = 1\,099\,511\,627\,776$
péta	Po	10^{15}	13%	pébi	Pio	2^{50}
exa	Eo	10^{18}	15%	exbi	Eio	2^{60}
zétta	Zo	10^{21}	18%	zébi	Zio	2^{70}
yotta	Go	10^{24}	21%	yobi	Yio	2^{80}

ATTENTION : En anglais “B” fait référence à l’**octet** traduit par *Byte*

ex : KB ou kB, MB, GB, TB, KiB, ...

(En français, kb fait référence au kilo bit !!!)

La mémoire : une ressource importante

RAM = mémoire principale qui doit être gérée avec attention !

On l'aimerait : grande, "rapide", pas cher !

Problème : les technologies actuelles ne le permettent pas

Solution : hiérarchisation de la mémoire

- quelques Mo de mémoire cache volatile, rapide et chère
- quelques Go de mémoire de rapidité d'accès et de prix moyens
- quelques To de mémoire de masse plus lente et non volatile

Quel est le rôle du système d'exploitation ?

Coordonner la manière dont ces mémoires sont utilisées via le **gestionnaire de mémoire**

Mémoire vive ou RAM (quelques Go)

La mémoire vive (ou RAM) est la mémoire dans laquelle les données sont placées lors de leur traitement

Quelques caractéristiques de la RAM :

- mémoire à accès rapide
- mémoire volatile *i.e.*, besoin de courant électrique

- "cases" numérotées à partir de 0
  - ↪ le numéro correspond à l'adresse
  - ↪ chaque case contient un octet (8 bits, 1bit : 1 ou 0)
    1 octet = 8 bits ⇒ $2^8 = 256$ valeurs possibles
- Deux types d'opérations possibles : lecture/écriture

Représentation des données en mémoire

Comment stocker et representer des entiers, des réels, des chaînes de caractères, des programmes, etc. ?

- entiers non signés : codage en base 2 en binaire
  - sur 1 octet : $0 \rightarrow 255$ ($00 \rightarrow FF$ en hexadécimal)
  - sur 4 octets : $0 \rightarrow 2^{32} - 1$ ($00000000 \rightarrow FFFFFFFF$)
- entiers signés : on utilise un bit de signe ou le complément à 2
  - 1 octet : $-128 \rightarrow 127$ ($80 \rightarrow 7F$)
- flottants : sur 4 ou 8 octets (*c.f.* cours Architecture S3)
- chaînes de caractères : la fin de la chaîne est toujours un octet nul
  - codage ASCII : 1 caractère = 1 octet
    Le premier bit est toujours à 0 (128 caractères possibles)
    'A' $\longrightarrow$ 0100 0001 (41 en hexa) 'B' $\longrightarrow$ 0100 0010 (42)
    'a' $\longrightarrow$ 0110 0001 (61) 'b' $\longrightarrow$ 0110 0010 : (62)
  - codage iso-8859-1 ou latin 1 : rajoute 128 caracteres
  - UTF-8 : sur 1 à 4 octets

Ex : "Baa" $\Rightarrow$ en ASCII ou latin-1 : 42 61 61 00

Hiérarchie de mémoire

Différents types de mémoire (on souhaite grande, rapide, pas cher)

- avec différentes caractéristiques :

  volatile ou pas, temps d'accès, débit, taille, prix/taille, ...

- du plus cher et plus rapide, au moins cher et plus lent :

	tps d'accès (cycles)	débit	taille typique
registres de processeurs	$\simeq 1$	-	20×4 octets
cache niveau 1	$\simeq 4$	qq Go/s	128 Ko
cache niveau 2	$\simeq 10$	idem	1 Mo
cache niveau 3	$\simeq 20$	idem	6 Mo
RAM	$\simeq 100$	idem	8 Go
Disque dur SSD	$\simeq$ 1ms	500Mo/s	250Go
Disque dur	$\simeq$ 10 ms	100Mo/s	1 To

Une note sur les prix du Gigaoctet

En 2015, RAM : 10 euros/Go ; SSD : $0,27$ euros/Go ; DD : $0,05$ euros/Go

Évolution des prix du Go (DD)

- 1956 : 1 Go = 26 000 000 euros/Go !!!
- 1973 : 1 Go = 3 000 000 euros/Go
- 1980 : 1 Go = 100 000 euros/Go
- 1987 : 1 Go = 40 000 euros/Go
- 1995 : 1 Go = 800 euros/Go
- 2002 : 1 Go = 2 euros/Go
- 2007 : 1 Go = $0,28$ euros/Go
- 2015 : 1 Go = $0,05$ euros/Go

Utilisation mémoire dans système multitâche

Système multitâche $\Rightarrow$ Plusieurs processeurs en RAM

Trois types de problèmes

- Isolation de processus
- Placement des processus
- La taille de la RAM est limitée

N.B. : Aucun problème s'il n'y a qu'un seul processus en RAM

Problème d'isolation de processus

Un processus ne doit pas pouvoir accéder à la RAM d'un autre processus ou du système d'exploitation

Comment ?

- Le SE alloue une zone de RAM à un seul processus
- Déclenchement d'une erreur si un processus accède à la mémoire non allouée

Pourquoi ?

- Pour garantir un fonctionnement correct des processus ou du SE
- Pour garantir la confidentialité des données

Problème de placement des processus en RAM

Comment placer les processus dans la mémoire RAM ?

Solution la plus simple : Chaque processus utilise une zone contiguë de RAM

Cependant

- Comment le SE peut savoir combien de RAM un processus va avoir besoin ?
- Comment gérer la variation besoins ? (ex : ouverture d'un petit fichier, puis d'un gros)
- Que se passe-t-il s'il n'y a pas de place assez grande (mais plein de petites places) ?
- Que se passe-t-il s'il y a plusieurs places assez grandes ?

Remarque : il est en général impossible de déplacer un processus en RAM

Problème de taille de RAM limitée

Comment gérer le petit espace mémoire de la RAM ?

Pourquoi ?

car les processus en utilisent de plus en plus

Loi de Parkinson

Les programmes s'accroissent pour remplir la mémoire disponible qui leur est réservée

La mémoire virtuelle

La mémoire virtuelle

Mémoire qui est vue par le processus comme de la mémoire vive (RAM) et qui est en réalité située sur le disque dur

Objectif : régler les problèmes précédents

La mémoire virtuelle—Le principe

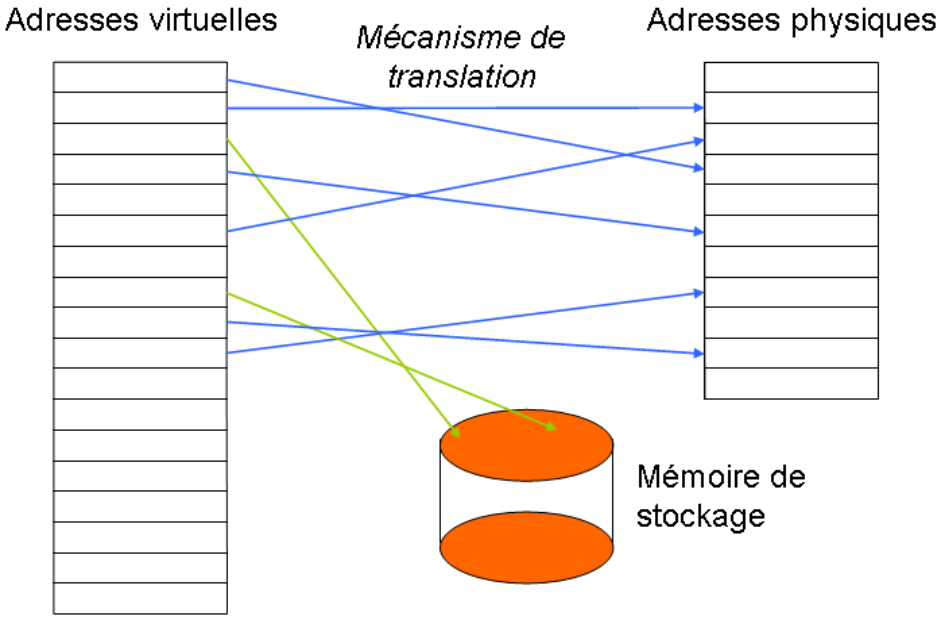
Les processus utilisent des adresses virtuelles qui sont ensuite traduites en adresses physiques par le matériel

Unité de gestion mémoire (*Memory Management Unit*-MMU)

La MMU est un composant qui permet de contrôler les accès qu'un processeur fait à la mémoire

Il permet de traduie les adresses (sorte de dictionnaire) :

$$(\text{processus}, @\text{virt}) \longleftrightarrow @\text{physique}$$



Mémoire virtuelle—Swap (va et vient)

Swap

Zone de mémoire sur le disque mise à disposition par le SE pour stocker une partie des données inutilisées par un processus

But : Utiliser de la mémoire disque pour faire "comme s'il" y avait plus de RAM

Principe : Lorsque le processus veut accéder à des données stockées dans le swap, le SE les remet en RAM en copiant d'autre données sur le disque

Avantage : libère de la RAM ("comme si" on avait plus de RAM)

Inconvénient : l'accès au disque est lent

- ⇒ ne fonctionne que si l'on a peu d'accès aux zones sur le disque
- ⇒ le SE ne doit pas se tromper quand il met des données sur le disque
- ⇒ il ne faut pas en abuser

Un type de mémoire virtuelle : la pagination

- La mémoire virtuelle est découpée en pages (virtuelles)
- La mémoire physique est découpée en cases (physiques)
- Les pages et les cases ont une taille P fixe
- La MMU traduit les adresses virtuelles en adresses physiques

Remarques

- À chaque page virtuelle correspond une case physique ou une erreur
- Certaines pages ne sont pas en RAM (pas allouée ou sur le disque)

$\Rightarrow$ Bit de validité

Pagination : Table des pages élémentaire

Table des pages

"Dictionnaire" qui indique la correspondance : **(processus,page)** $\longleftrightarrow$ **case**

- Un table par processus
- Informations supplémentaires
  - Présent/absent en mémoire (pour le swap)
  - Protection : accessibilité en lecture/écriture/exécution
  - Modifiée et/ou référencée (pour l'algo de remplacement)
- Changement de contexte $\Rightarrow$ commutation de la table des pages
- Trop lourd pour être dans la MMU

$\Rightarrow$ elle est en mémoire RAM avec juste un registre pointant dans la MMU

$\Rightarrow$ tout accès par une adresse virtuelle requiert 2 accès en RAM

Pagination : Table des pages élémentaire

Exemple avec 1 processus (pages et cases de 1000 octets)

```
RAM : C0 C1 C2 C3 C4 C5 C6  C7 C8 C9
      P3 -- P0 P1 -- -- P20 -- -- --

proc : P0 P1 P2 P3 P4 P5 P6 ..... P20
       code    |D | non alloué  |pile
             P2 sur disque
```

certaines pages ne sont pas en RAM $\Rightarrow$ bit de validité

Pagination : Table des pages élémentaire

Exemple avec 2 processus (avec partage) : Deux tables des pages

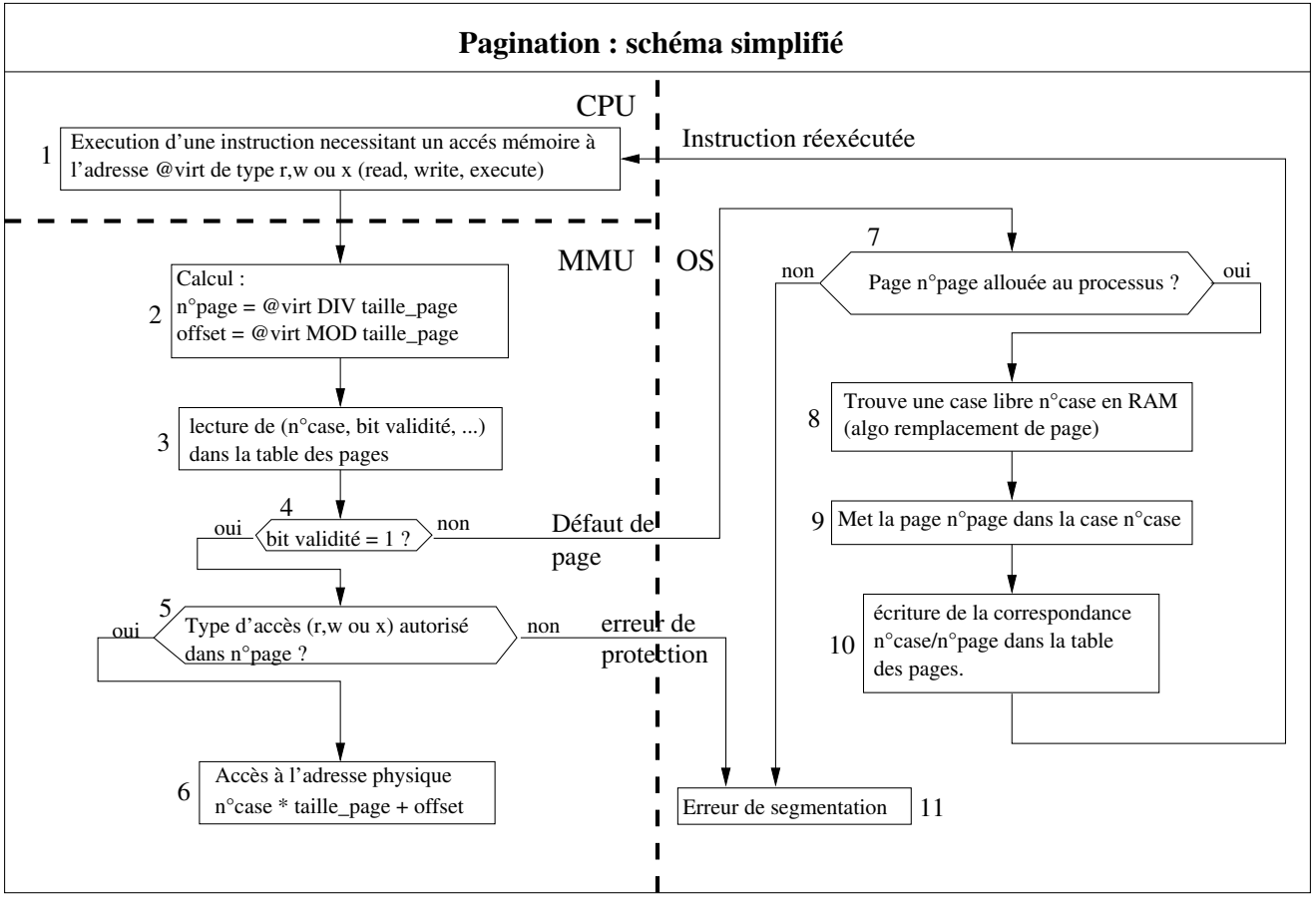
```
                data                pile
proc2:        P0 P1 P3 P4         P19 P20
RAM : C0 C1 C2 C3 C4 C5 C6  C7 C8 C9
proc1:P3      P0 P1       P20

proc : P0 P1 P2 P3 P4 P5 P6 .....  P20
       code     |D | non alloué  |pile
              P2 sur disque
```

- Partage de C2 et C3 $\Longrightarrow$ Une seule copie du code du programme en RAM
- Pour garantir l'isolation : les processus ne doivent pas pouvoir écrire dans les pages correspondantes
  - bit de droit d'accès dans la table des pages
  - un bit d'accès (mis à 1 lors d'un accès à la page)
  - un bit de modification (mis à 1 lors de la modification) *(dirty bit)*

Pagination : Algorithme de traduction (fait par la MMU)

L'algo de traduction est exécuté pour les pages dont le bit de validité vaut 1

$\Longrightarrow$ Les trois étapes du schéma : 2-3-6



Pagination : Algorithme de traduction

Exemple avec la table des pages suivantes

```
n_page : 0 1 2 3  4 5 6 7 8 9 ...
n_case : 3 2 5 11 0 9 8 7 6 4 ...

Pages de 1000 octets, traduire l'adresse virtuelle @v = 8083 :
     n_page = 8083 /1000 = 8
     d = 8083 mod 1000 = 83
     n_case = 6
     @p = 6*1000 + 83 = 6083

Pages de 1024 octets, traduire @v = 2100 :
    n_page = 2100 / 1024 = 2  ==> n_case = 5
    d = 2100 mod 1024 = 64
    @p = 5*1024 + 64 = 5120+64 = 5184
```

Remarque :

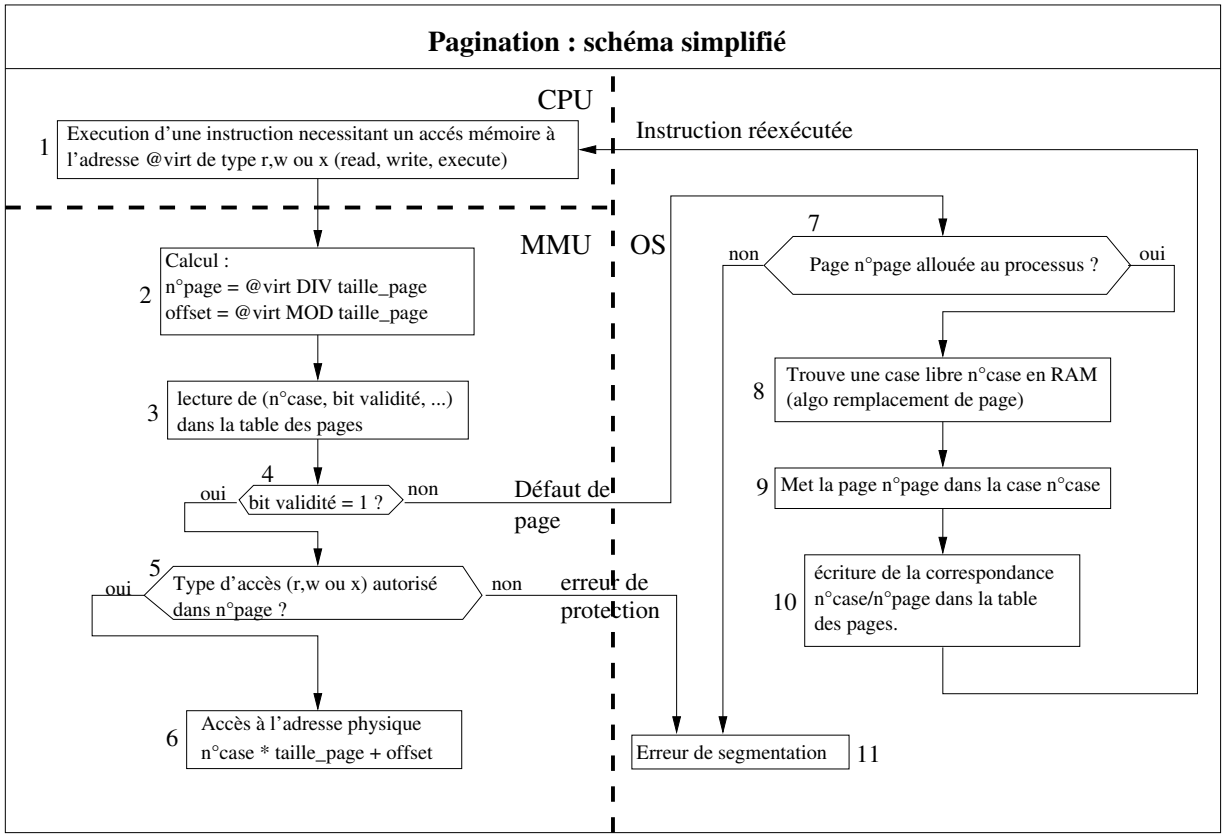
en base 10, c'est facile si T est une puissance de 10

en base 2 (ou 16), c'est facile si T est puissance de 2

Pagination : Défaut de page

Étape 4. Si le bit de validité est à 0 dans la TDP, il n'y a pas de traduction car soit la page n'est pas allouée au processus, soit elle est sur le disque

⇒ Déclenchement d'un "défaut de page"
↪ Le processus en cours est interrompu et le SE reprend la main



Pagination : Algos de remplacement de pages

Pourquoi ?

En cas de défaut de page, les algorithmes de remplacement permettent de déterminer une case physique victime qui sera sauvée sur le disque (si nécessaire) et remplacée par la page virtuelle à laquelle on souhaite accéder

Algorithme de Belady

- La victime est la page qui sera référencée le plus tard dans le futur

↪ **optimal, mais impossible à mettre en œuvre**

↪ il faut connaitre à l'avance les accès

FIFO : First In First Out

- La victime est la page la plus anciennement chargée

- Mémorise dans une file FIFO les pages présentes en mémoire

$\Rightarrow$ Supprime la page en tête de file

$\hookrightarrow$ Rarement utilisé car il induit souvent des défauts de pages

Pagination : Algos de remplacement de pages

NRU : Not Recently Used

- La victime est la page "la plus" non récemment utilisée

- S'il existe des pages non référencées ($A = 0$) et non modifiée ($M = 0$)
  alors il sélectionne une page et la retire

- Sinon, s'il existe des pages telles que $A = 0$ et $M = 1$
  alors il sélectionne une page et la retire (et sauvegarde de la page sur le disque)

- Sinon, s'il existe des pages telles que $A = 1$ et $M = 0$
  alors il sélectionne une page et la retire

- Sinon, s'il existe des pages telles que $A = 1$ et $M = 1$
  alors il sélectionne une page et la retire (et sauvegarde de la page sur le disque)

où A =bit d'accès et M =bit de modification

Pagination : Algos de remplacement de pages

LRU : Least Recently Used

- La victime est la page la moins utilisée
- Utilisation d'une liste chaînée de toutes les pages en mémoire
  - La plus utilisée est en tête de liste et la moins utilisée est en queue

↪ Coûteux et difficile à implémenter

Approximation de LRU : l'algorithme de l'horloge (ou de la seconde chance)

- Liste circulaire en forme d'horloge avec un pointeur sur la plus vieille page
  - Lorsqu'il y a défaut de page, la page pointée est examinée
    - Si $A = 0$ (le bit d'accès de la page est à 0)
      alors elle est retirée, la nouvelle page est insérée à sa place (et son bit d'accès est mis à 1) et le pointeur avance
    - Sinon, si $A = 1$
      alors $A = 0$ et le pointeur avance
  - Lorsqu'on accède à page présente dans la liste, son bit d'accès A est mis à 1

↪ Simple d'implémentation

Organisation de la mémoire pour les processus

4 zones mémoires sont utilisées par un processus

1. **Texte** : code du programme (y compris les bibliothèques)
2. **Données** : variables globales ou constantes (de taille fixe)
3. **Tas** : Données de taille variable (`malloc`)
4. **Pile** : Var locales des fctions, paramètres, gestion des appels de fctions

1 et 2 : taille fixée

↪ Allouée par le SE au moment du lancement du processus

3 et 4 : taille variable pendant l'exécution du processus

↪ Le processus demande au SE s'il veut plus de RAM (`malloc`) ou moins (`free`)

Entrées/Sorties

Rôle du SE dans la gestion des Entrées/Sorties

Le Système d'exploitation

- **Contrôle** les périphériques d'E/S
  - Envoi des commandes aux périphériques, traitement des interruptions et des erreurs
- Fournit une **interface simple** pour l'accès aux périphériques
  - Le plus possible indépendante du matériel

Exemples de périphériques d'E/S

Disques durs, lecteurs de CD/DVD/Bluray, carte réseau, horloge, clavier, souris, carte graphique, ...

N.B. On ne s'intéresse pas à l'aspect physique (électronique ou mécanique) mais plutôt à l'interface logicielle : commandes acceptées par le périphérique, ses fonctions et les erreurs qu'il peut signaler

Les périphériques d'E/S

Les périphériques d'E/S sont classés en 2 catégories

1. Les périphériques blocs *(block devices)*
2. Les périphériques caractères *(character devices)*

Remarque :
Certains périphériques ne rentrent ni dans un groupe ni dans l'autre
(ex : l'horloge qui génère des interruptions)

Les périphériques d'E/S

Les périphériques blocs

- Transmettent ou reçoivent les information sous forme de paquets d'octets de tailles fixe

↪ Les information sont stockées dans des blocs de tailles fixe

↪ Chaque bloc est repéré par un numéro

↪ Chaque bloc peut être lu ou écrit indépendamment des autres

- On doit écrire/lire un bloc en entier

Exemples : Disque dur, disquette, DVD, ...
En gros : tous les périphériques de stockage

Les périphériques d'E/S

Les périphériques caractères

- Transmettent ou reçoivent des informations octet par octet

↪ Les informations ont la forme d'un flux de données

Exemples : Réseau, souris, clavier, imprimante, carte graphique (dans certains cas), carte son, ...

Contrôleur de périphériques

Le sytème d'exploitation ne contrôle généralement pas directement les périphériques
↪ Le SE communique avec le contôleur de périphériques

Contrôleur de périphériques

Matériel qui commande le périphérique d'E/S

Le SE envoie des commandes et des données au contrôleur et récupère les données

Les contrôleurs de périphériques peuvent utiliser des interruptions pour signaler la survenue de certains événements

Exemple : nouvelles données (clavier, souris), données demandées disponibles (disque), périphérique prêt à recevoir de nouvelles données (réseau), ...

Le Système de gestion de fichiers

Le Sytème de gestion de fichiers - Késako ?

Système de gestion fichiers

Le Système de Gestion de Fichiers (SGF) est la partie la plus visible d'une SE.
Le SGF se charge de gérer le stockage et la manipulation des fichiers sur le matériel physique, il offre également les primitives pour manipuler les fichiers.

Les tâches effectuées par le SGF :

- Fournit une interface "conviviale" pour faciliter la manipulation des fichiers
- Gestion de l'organisation des fichiers sur le disque (allocation de l'espace disque aux fichiers)
- Gestion de l'espace libre
- Gestion des fichiers dans un environnement multi-utilisateurs

Organisation d'un disque

- Un MBR : *Master Boot Record*
- Des partitions

Le MBR

Le MBR est le nom donné au premier secteur adressable d'un disque dur
Il contient la table des partitions et le code d'amorçage boot

Les partitions

Une partition est un fractionnement d'un disque dur réel (matériel) en plusieurs disques virtuels (logiques)
Chaque partition contient un système de fichiers
C'est un "superbloc"

Le Formatage

Formatage

Pour qu'un système de fichiers puisse gérer des fichiers sur une unité de stockage, son unité doit être **formatée** (selon les spécificités du SGF)

Le formatage inspecte les secteurs, efface les données, crée le répertoire racine du système de fichiers et crée un "superbloc" pour stocker les informations nécessaires pour assurer l'intégrité du système

Remarque : Le "superbloc" contient notamment :

- l'identifiant du système de fichiers (`C:`, `D:`, ...)
- le nombre de blocs dans le système,
- la liste des blocs disponibles,
- l'emplacement du répertoire racine,
- la date et l'heure de la dernière modification du système,
- une information indiquant s'il faut tester l'intégrité du système

Notion de fichiers

Fichier

Un fichier est l'unité de stockage logique mise à la disposition des utilisateurs pour l'enregistrement de leurs données : c'est l'unité d'allocation

Remarque :
Le SE fait la correspondance entre le fichier et le système binaire utilisé lors du stockage de manière transparente pour les utilisateurs

Les fichiers en eux-mêmes

Noms des fichiers

- Sous Windows : est un attribut du fichier
- Sous Linux : est associé à un lien sur le fichier
  - ⇒ un fichier peut avoir plusieurs noms/liens

Atributs des fichiers

- **Ex :** UID, GID, permissions, dates (dernière modification), taille type (fichier, repertoire, lien symbolique, périphérique)

↪ Affichage des attributs : `ls -l` ou `stat`

```
-rw-r--r--   1 Milie  staff       0 19 fév 15:21 test
```

- Sous Linux : numéro du fichier (unique) : numéro d'i-nœud (*i-node*)

↪ Affichage de l'i-nœud : `ls -i` (`39329029 test`)

Opérations sur les fichiers

- Création, suppression, lecture écriture, modification des attributs

Les Répertoires

Répertoire

Un répertoire *(directory)* est un fichier dont le contenu est une liste de descriptions de fichiers. Il contient les liens physiques vers d'autres fichiers ou répertoires.

- Un répertoire a donc les même types de propriétés qu'un fichier
- Les opérations possibles : création, suppression (si vide), ajout d'un lien vers un autre fichier/repertoire

Arborescence

Chaque fichier ou répertoire est référencé par un autre répertoire, ce qui forme une hiérarchie cohérente, appelée aussi arborescence, dont le point d'entrée est le répertoire racine. La racine est unique sur un système de type UNIX (/), ou sur une partition Windows (par exemple `C:`).

La structure hiérarchique des répertoires

Type Windows

- Plusieurs arborescences séparées
- Une (ou plusieurs) arborescence par périphérique
- La structure logique reflète la structure physique

Système de fichiers virtuel (UNIX)

- Une seule arborescence

⇒ Notion de montage d'un système de fichier (`mount`, `umount`)
  ↪ greffer la racine d'une arborescence non encore montée en un point accessible depuis la racine absolue

⇒ Retrouver le périphérique est plus compliqué (pour le SE)

- Les péripheriques sont des fichiers dans `/dev/`
  ↪ (`/dev/pts/X`, `/dev/hda`, `/dev/sda`, `/dev/mouse`, `/dev/keyboard`, ...)

N.B. : Pour pouvoir accéder à un fichier ⇒ Chemin d'accès (chemin absolu, chemin relatif, `.`, `..`, `~user`, `~/`)

Les liens

Principe

Un seul fichier auquel on peut accéder via différents chemins

Deux types de liens :

- Liens symboliques
- Liens physiques (ou liens durs)

Les liens symboliques

Lien Symbolique

Fichier spécial contenant un chemin d'accès à un fichier pointant sur un nom de fichier

Inconvénient : Le lien devient invalide si on déplace/renomme/supprimer la destination du lien

Exemples

- Sous Windows : les raccourcis
- Sous Linux : liens créés avec la commande `ln -s`

Les liens physiques

Lien Physique

Permet de donner plusieurs noms/chemin d'accès à un même fichier en pointant sur un numéro de fichier (sous Linux : les fichiers sont enregistrés sur la base d'un numéro)

Différences entre Linux et Windows

- Sous Windows : le nom est associé au fichier
- Sous Linux : le nom est associé au lien entre le répertoire et le fichier
  - ⇒ On peut créer plusieurs liens sur le même fichier (via `ln`)

Inconvénients

- Ils ne peuvent pointer que vers un élément du même système de fichiers
- Un seul lien pour les répertoires (hormis `.` et `..`)
- Il est impossible de distinguer le fichier orignal (ou maître) de ses références

Avantage

- Insensible au déplacement/renommage/suppression
  - → cela n'affecte que les liens pas les fichiers eux-même

Les Buffers (valable pour tout périphéques blocs)

Buffer

Une zone de RAM ou le système d'exploitation met en attente des données venant des périphériques ou allant vers les périphériques

Objectifs

- Simplifier l'accès aux périphériques blocs (écriture/lecture d'une partie de bloc)
- Améliorer les performances

Les fichiers sont lus par blocs puis mis dans des buffers en RAM par le SE
⇒ Les processus accèdent à ces copies en RAM
⇒ Peut entrainer des problèmes de synchronisation des modifications
↪ commande `sync` (faîte pas `umount`)

N.B.: Donc dans les cases de la RAM, il y a
le système d'exploitation, les pages de processus, les buffers, des cases libres

Les Buffers — Exemple

```
buffer avec 3 cases (blocs et cases de 1ko)
P lit octets 0 à 100 dans fich F => un acces disque  \
P lit octets 101 à 200 dans F => lect dans buffer    | dans buffer
...                                                  /
P lit octets 1001 à 1100 => lect dans buffer puis un acces disque
  puis lect dans buffer
...
```

écriture

S'il n'y a plus de place dans un buffer pour y mettre des blocs

- Le SE agrandit le buffer
- Le SE supprime libère un bloc du buffer (meme algo que remplacement de page)

Implantation des systèmes de fichiers

sous LINUX : 12 Millions de lignes !

Organisation en couches

a. Gestionnaire de système de fichiers virtuels
b. Gestionnaires de systèmes de fichiers (FAT, ext2, NTFS, ISO9660, NFS, ...)
c. Gestion des buffers
d. Pilotes disques/cdrom/clé USB — Pile réseau
e. Pilotes bus IDE, SCSI, USB ...
f. Matériel (contrôleurs disques, réseau)

N.B. : Un bus est un dispositif de communication partagé entre plusieurs composants d'un système

Gestionnaire de système de fichiers virtuels

Fournit :

- Fonctions de création/suppression de fichiers/répertoires
- Fonctions de lecture/écriture dans un fichier/répertoire

$\hookrightarrow$ Ces fonctions utilisent des chemins (absolus ou relatifs) pour désigner les fichiers/répertoires

Rôle :

- Contrôle les droits d'accès
- Doit déterminer à partir des chemins sur quel périphérique se trouve un fichier donné et quel système de fichier est utilisé

$\Rightarrow$ Contacte alors le bon gestionnaire de système de fichier

Gestionnaires de systèmes de fichiers

Fournit :

- idem que pour le gestionnaire de système de fichiers virtuels mais seulement pour un système de fichier particulier (ex FAT, ext2, ISO9660, NFS, ...)

Rôle :

- Contrôle dans quels blocs du disque sont stockés les fichiers

N.B. : il y a donc plusieurs gestionnaires

Gestion des Buffers

Fournit :

- Accès en lecture/écriture des blocs

Rôle :

- Verifie si les données sont sur le disque ou en RAM

$\Rightarrow$ Déclenche les accès disque

Pilotes disques/cdrom/clé USB —- Pile réseau

Fournit :

- Accès en lecture/écriture des blocs numérotés de taille fixe
- Basiquement : une fonction `lire_bloc(int num)` et une `ecrire_bloc(int num)`

Rôle :

- Met les disques en veille ou les réveille en fonction des besoin
- Traduuit ces opérations en messages à envoyé sur le bus
- Rapporte les erreur matérielles

Pilotes bus IDE, SCSI, USB ...

Fournit :

- Fonctions pour communiquer avec les contrôleurs des périphériques (pour l'envoi et la réception de messages)

Rôle :

- Envoie les messages aux contrôleurs des périphériques

Exemple :

```
fich=open("/home/toto/boum")  => pas acces disque
fprintf(fich,"hop")           => acces disque
fprintf(fich,"badaboum")      => buffer
```

Introduction aux Réseaux

Utilité des réseaux

- Partage de ressources
- Fiabilité / sécurité
- Communication

Utilité — Partage de ressources

- Données (fichiers, bases de données,. . .)
- Applications
- Puissance de calcul
- Périphériques (imprimantes, scanner, . . .)

⇒ Utilise souvent le modèle client/serveur

- Serveur : celui qui propose la ressource
- Client : celui qui la demande

↪ le client envoie une demande et le serveur répond

Utilité — Fiabilité et sécurité

- Duplication de données

↪ permet d'éviter une perte accidentelle ou par malveillance

- Duplication d'unités de calcul

↪ cas critique : centrale nucléaire, avion, gestion trafic aerien, commerce en ligne, ...

Utilité — Communication

- E-mail
- Messagerie instannée (chat) (*e.g.*, Skype, Facebook Messenger, Snapchat, . . .)
- Forums
- Téléphonie IP
- Visio-conférence
- Partage de fichiers
- . . .

Caractéristiques physiques

Techniques de transmission

- **Par diffusion**
  - Un seul canal de transmission pour toutes les machines
  - Ce qu'envoie une machine est reçu par toutes les autres

Inconvénient : Si deux machines émettent en même temps ⇒ Collision

⇒ Réservé aux petits réseaux

- **Point à point**
  - Machines reliées deux à deux
  - Réseau représenté par un graphe

Inconvénient : Si deux machines ne sont pas reliées : il faut passer par des machines intermédiaires (il peut y avoir plusieurs possibilités)
⇒ Choisir un chemin : routage

Caractéristiques physiques

Topologie d'un réseau

Topologie ⇔ la "forme" du réseau, son organisation
Toplogie physique ⇔ interconnexion réelle des équipements
Topologie logique ⇔ simulation d'une interconnexion par logiciel ou via des équipements particuliers

Généralement un réseau est représenté par un graphe :

- Sommets : machines
- Arêtes : canaux de trasmission (fibre optique, wifi, ...) de débits variables

Deux types de machines :

- machines dédiées au routage des messages : routeurs

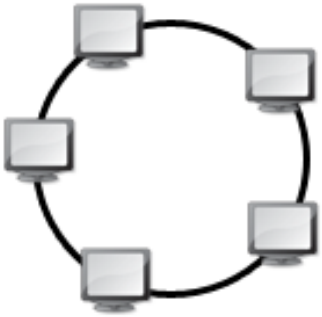
↪ Routage : pr envoyer un message il faut trouver et choisir un chemin

- les autres

N.B. Adressage : pour communiquer chaque machine doit avoir une adresse

Quelques topologies classiques

En anneau



Bus

Hiérarchique

Étoile

Maillé

Images issues de Wikimedia Commons (par Yannickv-licence GPL)

Conception d'un réseaux - les problématiques

Les problèmes auxquels il faut s'intéresser lors de la conception sont :

- Le débit entre les machines
- Le temps de transit des messages
  - ↪ Combien de temps met un message pour aller de A à B
- La résistance aux pannes (routeurs ou de canaux de transmission)
- Le coût

Aspects Logiciels

Service en mode connexion et sans connexion

- Sans connexion
  - envoi de messages
  - pas forcément fiable
  - pas de garantie de l'ordre d'arrivée par rapport à l'ordre d'émission

  $\Rightarrow$ Similaire à "La Poste"

- Mode connexion (*e.g.* tel)
  - flux de données $\simeq$ tuyau
  - fiable, ordre garanti
  - protocole : établissement de la connexion $\rightarrow$ communication $\rightarrow$ fin transmission

Aspects Logiciels

À résoudre

- identification source et destination (adressage)
- choix du chemin (routage)
- contrôle d'erreur (détection, correction)
- gestion de la différence de vitesse entre émetteur/récepteur
- les messages n'arrivent pas forcement dans l'ordre d'émission
  $\Rightarrow$ il faut les réordonner (mode connexion)
- tous les protocoles n'acceptent pas toutes les tailles de messages
  $\Rightarrow$ il faut découper / réassembler les messages

Aspects Logiciels

Logiciel organisé en couches

Chaque couche utilise les services de la couche inférieure et fournit des services à la couche supérieure.

Sur chaque couche

- on rajoute un en-tête/en-queue
- on découpe éventuellement en sous-messages plus petits

Exemple : avec 3 couches

```
message =>
        => et1:message
        => et2:et1:mess  et2:age  (découpage en 2)
        => et3:et2:et1:mess:eq3 et3:et2:age:eq3   (avec en queue)
        => réseau
        => réassemblage
```

Modèle TCP/IP

TCP/IP

TCP/IP (Transmission Control Protocol/Internet Protocol) est une suite de protocoles représentant l'ensemble des règles de communication sur internet et se basant sur la notion d'adresse IP.

Caractéristiques :

- Fractionnement des messages en paquets
- Utilisation d'un système d'adresses
- Acheminement des données sur le réseau (routage)
- Contrôle des erreurs de transmission de données

Le Modèle TCP/IP — un modèle en couche

Les 4 couches du mocèles TCP/IP

- Couche application → applications standards du réseau (smtp, ftp, . . .)
- Couche transport (TCP) → assure l'acheminement des données (et l'état)
- Couche internet (IP) → fournit le paquet de données
- Couche accès réseau (physique) → spécifie la forme des données

Intérêt des couches

Séparer le problème en différentes parties selon leur niveau d'abstraction

Chaque couche

- communique avec une couche adjacente (celle du dessus ou celle du dessous)
- utilise les services des couches inférieures
- fournit des services à celle de niveau supérieur

Le Modèle TCP/IP — Encapsulation des données

Au niveau de la machine émettrice :

↪ À chaque couche, un **en-tête** est rajouté pour garantir la transmission

Au niveau de la machine réceptrice :

↪ À chaque couche, l'**en-tête** est supprimé

⇒ À la reception le message est donc dans son état d'origine

À chaque couche, le paquet change d'aspect et donc de nom :

- Couche application : **message**
- Couche transport : **segment**
- Couche internet : **datagramme**
- Couche accès réseau : **trame**

Modèle TCP/IP

La couche accès réseau—Première couche de la pile TCP/IP

C'est la couche qui permet d'accéder à n'importe quel réseau physique
Elle contient toutes les spécifications concernant la transmission de données

Elle prend en charge

- L'acheminement des données sur la liaison
- La coordination de la transmission de données (synchronisation)
- Le format des données
- La conversion des signaux (analogique/numérique)
- Le contrôle des erreurs à l'arrivée

Modèle TCP/IP

La couche internet—La plus "importante"

Elle définit les datagrammes et gère les notions d'adressage IP (une machine est associée à une adresse IP) et du routage
Elle permet l'acheminement des datagrammes vers des machines distantes et gère leur fragmentation et leur assemblage (à la réception)

- Pas de garantie sur l'ordre d'arrivée des datagrammes, ni même de garantie qu'ils arrivent
- le datagramme contient, en plus des données, les adresses d'origine et de destinations, etc.

N.B. La couche contient 5 protocoles : IP, ARP, ICMP, RARP et IGMP

Modèle TCP/IP

La couche transport

Elle permet à des applications tournant sur des machines distantes de communiquer

Problème : Sur une machine il peut y avoir plusieurs processus qui envoient /recoivent des paquets $\Rightarrow$ Comment les distinguer ?
Solution : Les ports $\rightarrow$ Identifiant (sur 16 bits) permettant d'associer un type d'application à un type de données

N.B. La couche contient 2 protocoles :

- TCP (transmission control protocol) : assure le contrôle des erreurs, fiable, mode connexion
- UDP : (user datagram protocol) : non fiable, mode sans connexion

Modèle TCP/IP

La couche application—Le sommet des couches

Elle contient les applications réseaux permettant de communiquer grâce aux couches inférieures

Les logiciels de cette couche communiquent grâce aux deux protocole de la couche transport : TCP/UDP

N.B. La plupart des applications sont des services réseau

⇒ Fournissent l'interface avec le SE

Par exemple, pour :

- la gestion (transfert) de fichier et d'impression
- la connexion au réseau
- la connexion à distance
- les utilitaires internet

Avant de finir, quelques mots sur le DNS

DNS

Le *Domain Name System* (DNS, système de noms de domaine) est un service utilisé pour traduire une adresse de type `www.univ-st-etienne.fr` (*Fully Qualified Domain Name*, FDQN) en adresses IP portant ce nom

Structure aborescente, par exemple :

```
www.univ-st-etienne.fr
=> fr (Top Level Domain)
   univ-st-etienne (domaine)
   www  (nom de machine)
```

```
moneta.c2m.univ-st-etienne.fr
=> fr (Top Level Domain)
   univ-st-etienne  (domaine)
   c2m (sous domaine)
   moneta (nom de machine)
```

Pour connaitre l'@IP correspondante à un nom

Requête au serveur de noms qui maintient une table de correspondance
S'il ne trouve pas le nom dans sa table, il requête un autre serveur de noms

Les serveurs de noms sont organisés en une arborescence $\Rightarrow$ Si un serveur ne sait pas répondre à une requete, il la fait passer au niveau supérieur